

6.2 实验 11 流水灯实验

6.2.1 实验目的

- 1. 熟悉并掌握开关和显示灯作为输入和输出设备进行各种实验的方法。
- 2. 掌握灵活运用 Verilog HDL 语言进行各种描述与建模的技巧和方法。

6.2.2 实验内容与原理

1、计算机的外设接口与读写过程

在计算机中，尤其是嵌入式系统中，开关是最常用的输入设备，而显示灯是最常用的输出设备，它们也是最简单的计算机外设。开关的外设接口一般采用数据缓冲器，而显示灯的外设接口则采用数据锁存器，如图 6.2 所示。

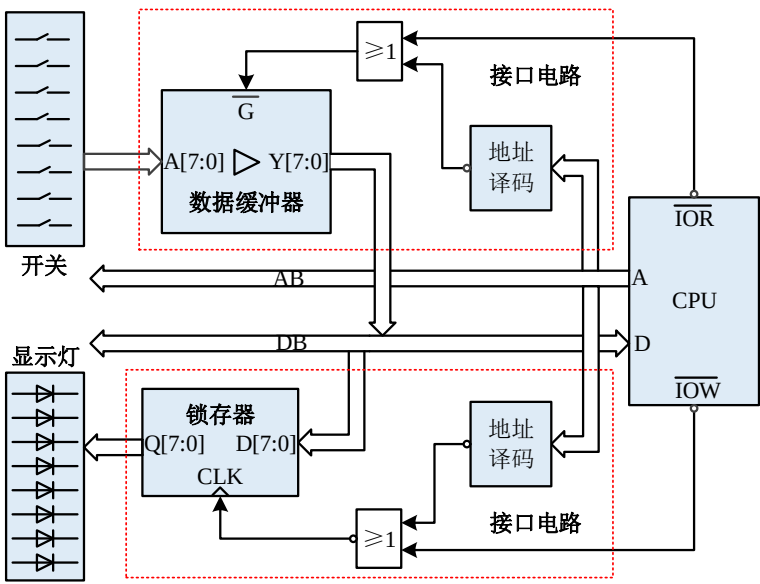


图6.2 开关和显示灯的接口电路

数据缓冲器和数据锁存器的区别在于：**数据缓冲器**是组合逻辑电路，只能支持数据通路的通断，而不能保持数据；但是**数据锁存器**是时序逻辑电路，在时钟上升沿锁存数据并保持到下次上升沿来临。

外设的地址一般称作**端口地址**，是指外设接口中寄存器或其他电路的地址。每个设备接口中均有一个专门的“地址译码”电路，用于设备的端口地址自我识别。CPU 的地址总线经过端口地址译码电路后，选中某个端口，CPU 发送读写信号后，访问选中的设备。

(1) CPU 读入外设数据

在图 6.2 中，如果 CPU 需要读入开关的数据，则在地址总线上送出开关的端口地址，然后发送 **I/O 设备的读信号 $\overline{IOR}$** ；这时，对于开关接口电路来说，地址译码电路识别出自己的端口地址，输出低电平的选通信号，与 CPU 发送的 $\overline{IOR}$ 信号经过或门，输出了数据缓冲器的使

能信号 $\overline{G}$ ，从而数据缓冲器接通，开关上的数据从缓冲器的 A 端送到 Y 端，送上数据总线，供 CPU 读取。

(2) CPU 写数据到外设

同理，如果 CPU 要将数据送到显示灯显示，则需要在地址总线上送出显示灯的端口地址，在数据总线上送出要显示的数据，然后发送 **I/O 设备的写信号 $\overline{IOW}$** ；与此同时，显示灯接口中的地址译码电路识别出自己的端口地址，输出低电平的选择信号，与 CPU 发送的 $\overline{IOW}$ 信号经过或非门，输出了数据锁存器的时钟信号 CLK 上升沿，这样就把数据总线上的数据锁存到了锁存器中，显示灯始终显示锁存器的输出端 Q 的数据。

(3) CPU 的 I/O 指令

那么，CPU 如何（何时以及依据什么）读取开关的数据或者输出数据到 LED 灯显示呢？由计算机的工作原理即可知：通过 CPU 执行**输入指令（IN）**来访问输入设备，而通过 CPU 执行**输出指令（OUT）**来访问输出设备。具体而言，CPU 执行输入指令时，将访问的输入设备（即开关）端口地址放在地址总线上，并在 $\overline{IOR}$ 信号上产生一个负脉冲；然后接口电路组合 $\overline{IOR}$ 信号和由地址译码电路产生的负电平，形成数据缓冲器的使能信号，输入（开关）数据便经数据缓冲器输出端稳定出现在数据总线上，CPU 则将其锁存到目的部件（一般是寄存器）中。计算机执行输出指令（OUT）时，将访问的输出设备（即 LED 灯）端口地址放在地址总线上，要输出的数据放在数据总线上，并在 $\overline{IOW}$ 信号上产生一个负脉冲；接口电路组合 $\overline{IOW}$ 信号和由地址译码产生的负电平，形成锁存器的打入脉冲；然后锁存器锁存数据总线上出现的待输出数据，并驱动发光二极管发光。

2、实验内容

本次实验尚没有 CPU 和地址译码电路，所以直接通过 FPGA 对开关和显示灯直接进行控制，原理如图 6.3 所示。

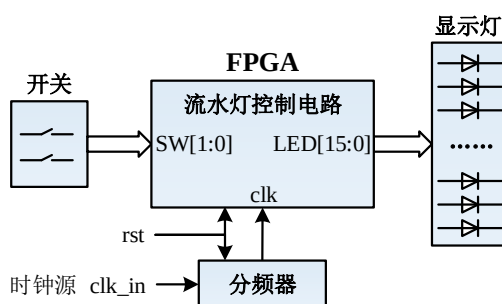


图6.3 流水灯实验原理图

实验要求根据读取的开关值 SW[1:0]，来控制 16 个 LED 显示灯以不同的方式流动显示：

- SW[1:0]=00：LED 显示灯从左到右逐个点亮，循环显示；
- SW[1:0]=01：LED 显示灯从右到左逐个点亮，循环显示；
- SW[1:0]=10：LED 显示灯从中间向两边逐个点亮，循环显示；
- SW[1:0]=11：LED 显示灯从两边向中间逐个点亮，循环显示；

3、流水灯的实现方法

为实现流水灯的功能，可以按照如下思路设计电路。

(1) 在 `rst` 信号有效时或者 `SW[1:0]` 的值改变时，根据 `SW[1:0]` 的值执行如下 LED 灯初始化赋值操作：

- `SW [1:0]=00`：要显示的初始值为 8000H。
- `SW [1:0]=01`：要显示的初始值为 0001H。
- `SW [1:0]=10`：要显示的初始值为 0180H。
- `SW [1:0]=11`：要显示的初始值为 8001H。

(2) 在每个 `clk` 上升沿来临时，读入开关 `SW[1:0]` 的值，并判断它和上次读入的开关值是否发生改变：

- 如果发生改变，按照步骤 (1) 重新赋值输出；
- 如果未发生改变，则将 LED 灯显示值进行不同的移位操作：
 - `SW [1:0]=00`：循环右移 1 位。
 - `SW [1:0]=01`：循环左移 1 位。
 - `SW [1:0]=10`：高 8 位循环左移 1 位，低 8 位循环右移 1 位。
 - `SW [1:0]=11`：高 8 位循环右移 1 位，低 8 位循环左移 1 位。

讨论三个问题：一是如何监测 `SW` 开关是否改变呢？可以定义一个中间变量，在 `rst` 有效时将其赋值为 `SW` 的值；之后在每个 `clk` 的上升沿，将 `SW` 的值与该中间变量比较，得出“是否改变”结论，无论结论如何，都需要将 `SW` 的值赋值给中间变量，以便下次 `clk` 上升沿来临时进行比较。二是循环移位如何实现？因为是在 `clk` 上升沿执行移位操作，电路属于时序逻辑电路，应使用“`<=`”进行赋值，对 8 位变量 `Q[7:0]` 循环左移 1 位，可以用“`Q<={Q[6:0],Q[7]}`”来实现，每一位都同时赋值。

三是移位时使用的 `clk` 如何产生？频率多少？流水的 `clk` 由实验板卡的时钟源 `clk_in` 分频产生，参照“实验 10 定时与分频实验”设计实现分频器。需要注意：为稳定显示，LED 显示灯数据移位的变化频率不宜过快，流水的频率（`clk` 的频率）可以定为 1Hz~5Hz。

6.2.3 实验要求

1. 编程实现流水灯的控制模块，能根据模式开关控制 16 个 LED 灯进行规定的 4 种流水模式；允许在此基础上附加设计其他的流水模式，例如闪烁、停顿、交错显示等。
2. 设计实现分频器模块，并以两个模块为基础构造顶层模块，实现稳定的流水灯控制。
3. 为顶层模块配置 FPGA 管脚，完成板级验证，分析并记录实验结果；要求拨动模式开关就能按照新的流水模式重新开始执行流水显示，而不需要按下 `rst` 键。
4. 撰写实验报告，重点写出实验设计方案、逻辑程序、板级验证方案及对板级实验结果进行分析，得到有效结论；并力所能及回答或实践本实验的“思考与探索”部分。

6.2.4 实验步骤

1. 创建一个新工程，编写流水灯控制模块的程序代码，请注意考虑切换流水模式后，流水操作的完整性。可以根据需要对该模块进行仿真验证。
2. 编写分频器模块代码，产生合适的流水频率的时钟信号，请注意你所使用的实验板卡的时钟源频率（100MHz，E3 引脚引入）。
3. 构造顶层模块，调用并连接流水灯控制模块和分频器模块，分频器产生的时钟信号作为流水灯控制模块的 `clk`。
4. 新建管脚约束文件：
 - （1）`SW[1:0]`：配置到两个开关上；
 - （2）`LED[15:0]`：配置到 16 个显示灯上，**注意配置的左右顺序，与流水方向保持一致；**
 - （3）`clk_in`：配置到板卡的时钟源上，即 E3 引脚；
 - （4）`rst`：可配置到按键或者开关上。
5. 综合、实现后生成*.bit 文件，下载到实验设备的 FPGA 芯片中。
6. 板级实验：首先按 `rst` 复位，然后在板卡上拨动 `SW[1:0]` 对应的开关，观察 LED 灯显示的流水状态是否符合预期；切换流水模式时，注意观察流水操作是否完整，例如循环左移时，是否从最右边第一个灯开始亮起。

6.2.5 思考与探索

1. 你的流水灯的流动频率是多少？如何实现的？
2. 在实验中，流水灯的流动方向由两位开关 `SW` 决定的，可以视为手动控制模式；现要求添加一种自动控制模式：此时流水灯动态自动演示，按照四种流水方向自动切换，譬如先从左到右，流水完成后，又从右到左，然后从中间向两边，最后从两边到中间，循环往复。编程实现并添加这个自动控制模式，并调试。
3. 观察城市中的霓虹灯、灯光秀、电子广告牌等，一般会有闪烁、流动、渐变等多种展示模式，选择或者设计一种你喜欢并能在实验板卡的 LED 灯上展示的流水模式，实现它。
4. 你碰到了哪些问题，又是如何解决的？